# CHAPTER 1:- INTRODUCTION

## 1.1 Background

In today's digital world, people use mobile phones and the internet for online banking, shopping, communication, education, and many other activities. However, the increasing use of the internet has also increased cybercrime, such as phishing attacks, fake websites, fraudulent links, malicious QR codes, scam SMS messages, and fake emails. These attacks can trick users into sharing sensitive information such as passwords, OTPs, bank details, and personal information.

Many users are unable to identify whether a link, message, QR code, or email is genuine or fraudulent. Traditional security methods often depend on previously identified malicious websites, which may not detect newly created scam links. Therefore, there is a need for a system that can help users identify potential cyber threats before interacting with suspicious content.

AI Cyber Shield is an Android-based cybersecurity application developed to help users identify and understand different types of digital scams. The application uses Java for the Android frontend and Python Flask for the backend server. Machine Learning techniques, including the Random Forest Classifier, are used along with rule-based checks to analyse suspicious content.

The application allows users to scan URLs, QR codes, SMS messages, and emails. After analysing the submitted content, it displays a result such as SAFE, SUSPICIOUS, or DANGEROUS, along with a confidence score, possible threat indicators, and security advice.

Firebase Authentication is used for user registration and login, while Cloud Firestore stores user profiles, scan history, and cybersecurity awareness scores. Local storage is also used to keep recent scan records available when an internet connection is unavailable. In addition, the application includes a Cyber Awareness Challenge that helps users improve their knowledge of online safety through interactive quizzes.

The main aim of AI Cyber Shield is to make cybersecurity easier to understand and more accessible to everyday mobile users by combining threat detection, security warnings, scan history, and awareness training in a single application.

## 1.2 Objectives

The main objectives of the AI Cyber Shield project are as follows:

1. To develop an Android mobile application that helps users identify potential phishing attacks and digital scams.

2. To provide scanning facilities for suspicious URLs, QR codes, SMS messages, and emails.

3. To use Machine Learning and rule-based techniques to analyse suspicious content and classify it as SAFE, SUSPICIOUS, or DANGEROUS.

4. To display threat explanations, confidence scores, security indicators, and safety tips so users can understand the scan results.

5. To provide secure user registration and login using Firebase Authentication.

6. To store scan history and user information using Firebase Cloud Firestore, with local storage as a fallback when offline.

7. To prevent users from directly opening links identified as dangerous and provide appropriate warning messages.

8. To improve cybersecurity awareness through interactive quizzes, score calculation, and explanations of correct and incorrect answers.

9. To provide a simple, user-friendly, and modern interface that makes the application easy to use.

## 1.3 Purpose, Scope, and Applicability

### 1.3.1 Purpose

The purpose of the AI Cyber Shield project is to provide a simple and useful mobile security solution that helps users identify possible online threats before they interact with suspicious content.

The application is designed to reduce the risk of phishing attacks and digital scams by analysing links, QR codes, SMS messages, and emails. It combines Machine Learning algorithms with rule-based techniques to identify suspicious patterns and provide meaningful results to users.

The system also aims to educate users about cybersecurity through interactive quizzes and safety tips. Firebase is used to manage user authentication and store scan history, while local storage helps retain recent scan records when the internet is unavailable.

Overall, the purpose of this project is to improve online safety, increase awareness about cybercrime, and help users make safer decisions while using digital services.

### 1.3.2 Scope

The scope of the AI Cyber Shield project includes the following features:

1. User Registration and Login: Secure login using Firebase Authentication.
2. URL Detection: Checks links and identifies potential phishing threats.
3. QR Code Scanning: Scans QR codes and analyses extracted links.
4. SMS and Email Analysis: Detects suspicious messages, links, and scam patterns.
5. Machine Learning Detection: Uses a Random Forest Classifier and rule-based checks through a Flask backend.
6. Threat Results: Displays threat status, confidence score, reasons, and safety advice.
7. Scan History: Stores previous scan results using Firestore and local caching.
8. Cyber Awareness Challenge: Provides quizzes at beginner, intermediate, and advanced levels.
9. Profile and Awareness Score: Maintains user details and cybersecurity quiz scores.

The project helps users identify potential digital threats through a mobile application. However, its accuracy depends on the datasets, detection rules, and trained models. It is a security assistance tool, not a guarantee of complete threat detection.

### 1.3.3 Applicability

AI Cyber Shield can be useful for different categories of users who regularly use mobile phones and internet-based services.

1. Students: Students can learn about phishing attacks, fake websites, online fraud, and safe browsing practices.

2. General Mobile Users: People who receive unknown links, suspicious messages, or unfamiliar QR codes can use the application to check for potential threats.

3. Online Banking and Shopping Users: Users can analyse suspicious messages and links that claim to be related to bank accounts, payments, refunds, prizes, or online shopping.

4. Email Users: Users can check suspicious emails for possible impersonation, misleading subject lines, and fraudulent requests for sensitive information.

5. Cybersecurity Awareness Programs: The application can be used as a learning tool to explain common cyber threats and improve awareness through quizzes and security tips.

6. Educational Institutions: Colleges and training institutes can use the application as a demonstration of how Android development, Machine Learning, cloud databases, and cybersecurity techniques can work together.

The application is mainly intended for individual users who need assistance in identifying potential digital scams. It can also serve as an educational project for understanding the practical use of Artificial Intelligence and cybersecurity in mobile application development.

## 1.4 Achievements

The AI Cyber Shield project combines mobile application development, Machine Learning, cybersecurity analysis, and cloud services into one system. The major achievements of the project are as follows:

1. Android Application Development: A mobile application was developed using Java in Android Studio with a modern Material Design 3 interface.

2. Multiple Scanning Facilities: The application provides separate facilities to analyse URLs, QR codes, SMS messages, and emails for potential digital threats.

3. Machine Learning Integration: A Python Flask backend was developed to process submitted content using Scikit-learn, TF-IDF feature extraction, a Random Forest Classifier, and rule-based threat detection.

4. QR Code Integration: The ZXing Android Embedded library was integrated to scan QR codes and extract their contents for security analysis.

5. Firebase Integration: Firebase Authentication was implemented for user registration and login, while Cloud Firestore was integrated to maintain user profiles, scan records, and awareness scores.

6. Threat Classification: The system provides three main results: SAFE, SUSPICIOUS, and DANGEROUS, together with confidence information, threat indicators, and security advice.

7. Scan History and Local Storage: Recent scan records can be cached locally using Android SharedPreferences and Gson, while Firestore supports cloud-based storage and synchronisation.

8. Cyber Awareness Training: An interactive quiz system was included with three difficulty levels, answer explanations, and a score-based assessment of cybersecurity awareness.

9. Link Safety Mechanism: The application prevents direct opening of links identified as dangerous and displays a warning to help users avoid risky interactions.

10. Testing and Validation: The project documentation records successful Gradle builds and testing of the backend with SAFE, SUSPICIOUS, and DANGEROUS test cases.

## 1.5 Organisation of Report

The project report of "AI Cyber Shield – Intelligent Multi-Modal Phishing & Digital Scam Protection System" is divided into six chapters. Each chapter explains a different stage of the project, from its introduction to its final results and future improvements.

**Chapter 1: Introduction**

This chapter provides an introduction to the AI Cyber Shield project. It explains the background of the project, objectives, purpose, scope, applicability, and achievements. It also describes how the report is organised.

**Chapter 2: Survey of Technologies**

This chapter explains the technologies, programming languages, software, tools, and libraries used in the project. It describes the importance of Java, Android Studio, Python, Flask, Machine Learning, Firebase, Cloud Firestore, and the ZXing library, along with the reasons for selecting them.

**Chapter 3: Requirements and Analysis**

This chapter describes the problem definition, limitations of the existing system, and proposed solution. It includes functional and non-functional requirements, project planning and scheduling, Gantt chart, PERT diagram, hardware and software requirements, preliminary product description, and conceptual models of the system.

**Chapter 4: System Design**

This chapter explains the design and structure of the AI Cyber Shield application. It covers the basic modules, data design, database schema, data integrity and constraints, procedural design, logic diagrams, data structures, algorithm design, user interface design, security issues, and test case design.

**Chapter 5: Result and Discussion**

This chapter presents the results obtained after implementing and testing the application. It discusses the major features, including user registration and login, URL scanning, QR code scanning, SMS and email analysis, threat classification, scan history, and the Cyber Awareness Challenge. It also explains the observed test results and limitations of the system.

**Chapter 6: Conclusion and Future Work**

This chapter summarises the project, its objectives, and its achievements. It explains how AI Cyber Shield helps users identify potential phishing attacks and digital scams. It also describes future improvements, such as advanced Machine Learning models, multilingual phishing detection, automated threat monitoring, and community-based threat reporting.

# CHAPTER 2:- SURVEY OF TECHNOLOGY

The AI Cyber Shield project uses various software, programming languages, and libraries to develop a secure Android application for detecting potential phishing attacks and digital scams.

### 2.1 Java

Java is used to develop the Android application's frontend. It handles user interaction, URL scanning, QR code scanning, and displaying results. It was selected because it supports Android development and provides reliable application functionality.

### 2.2 Android Studio

Android Studio is the development environment used to build, test, and debug the Android application. It was selected because it provides tools for designing user interfaces and developing Android apps using Java.

### 2.3 Python

Python is used for Machine Learning and backend processing. It was selected because it is easy to use and supports many libraries for data analysis and threat detection.

### 2.4 Flask

Flask is used to create the backend REST API that connects the Android application with the Machine Learning model. It was selected because it is lightweight and suitable for developing APIs.

### 2.5 Machine Learning and Scikit-learn

Scikit-learn is used to implement the Random Forest Classifier for analysing suspicious content. It helps classify submitted content into SAFE, SUSPICIOUS, or DANGEROUS categories. It was selected because it provides useful Machine Learning algorithms.

### 2.6 Firebase Authentication

Firebase Authentication manages user registration and login using email and password. It was selected because it provides a convenient and secure authentication service.

### 2.7 Cloud Firestore

Cloud Firestore stores user profiles, scan history, and awareness scores. It was selected because it provides cloud-based data storage and allows data to be accessed across devices.

### 2.8 ZXing Library

The ZXing library is used to scan QR codes and extract their contents. It was selected because it supports QR code scanning in Android applications.

### 2.9 SharedPreferences and Gson

SharedPreferences stores small amounts of data locally, while Gson converts data between Java objects and JSON. They were selected to support local scan caching and data handling.

### 2.10 Retrofit and OkHttp

Retrofit and OkHttp are used for communication between the Android application and the Flask backend. They help send scan requests and receive analysis results over the internet.

# CHAPTER 3:- REQIREMENT AND ANALYSIS

## 3.1 Problem Definition

In today's digital world, users face cyber threats such as phishing websites, fake links, malicious QR codes, scam SMS messages, and fraudulent emails. Many users cannot easily identify these threats, which may lead to financial loss or theft of personal information.

The AI Cyber Shield project aims to develop an Android application that analyses suspicious content using Machine Learning and rule-based checks. It classifies the content as SAFE, SUSPICIOUS, or DANGEROUS and provides security advice.

The main problems are:

1. Phishing URL Detection: Identifying fake and suspicious website links.
2. QR Code Analysis: Detecting potentially harmful links extracted from QR codes.
3. SMS and Email Analysis: Identifying suspicious messages, scam patterns, and misleading links.
4. Threat Classification: Analysing submitted content and displaying understandable results.
5. User and Data Management: Managing user accounts, scan history, and cybersecurity awareness scores.

## 3.2 Requirements Specification

The system requirements describe the functions that the application must perform.

Functional Requirements:

- Users can register and log in securely.
- Users can scan URLs and QR codes and analyse SMS messages and emails.
- The system displays a threat category, confidence score, possible indicators, and safety advice.
- Users can view previous scan results and filter them by threat status.
- Users can attempt cybersecurity quizzes and view their awareness scores.

- Users can manage their profile and access stored information.

Non-Functional Requirements:

- The application should be easy to use.
- User information should be stored securely.
- The system should provide results within a reasonable time.
- The application should handle invalid input and network errors.
- The system should support reliable communication between the Android application and backend.

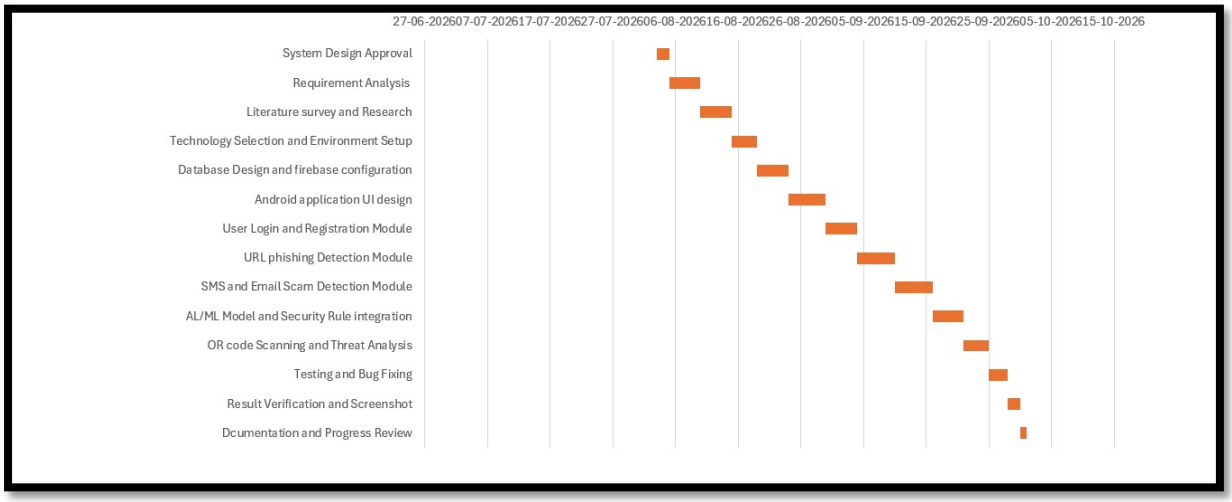
**Existing System and Its Problems:**

Traditional security systems often depend on known malicious websites or basic security rules. They may fail to identify newly created scam links and suspicious messages. Users may also need separate tools to check URLs, QR codes, SMS messages, and emails. AI Cyber Shield aims to provide these checking features in a single application.

### 3.3 Planning and Scheduling

Planning and scheduling divide the project into small tasks and arrange them in a suitable order. The project includes requirement analysis, application design, coding, backend development, Machine Learning integration, testing, and documentation.

**Gantt Chart:**



**Pert Chart:**




## 3.4 Software and Hardware Requirements

### 3.4.1 Hardware Requirements

| Hardware Component | Minimum Requirement |
|---|---|
| Processor | Intel Core i3 or equivalent and above |
| RAM | Minimum 8 GB recommended |
| Storage | At least 10 GB of free space |
| Display | Standard monitor with suitable resolution |
| Input Devices | Keyboard and mouse |
| Mobile Device | Android smartphone or emulator for application testing |
| Internet Connection | Required for Firebase services and backend communication |

### 3.4.2 Software Requirements

| Software Component | Description/Purpose |
|---|---|
| Operating System | Windows 10/11 or a compatible operating system |

| | |
|---|---|
| Android Studio | Used for Android application development and testing |
| Programming Language | Java for frontend and Python for backend development |
| Backend Framework | Flask for developing REST APIs |
| Machine Learning Library | Scikit-learn for implementing the Random Forest Classifier |
| Database and Authentication | Firebase Authentication and Cloud Firestore |
| QR Code Library | ZXing for QR code scanning |
| Communication Libraries | Retrofit, OkHttp, and Gson for API communication and JSON handling |
| Testing Tools | Android emulator, Android smartphone, and API testing tools if required |

### 3.5 Preliminary Product Description

AI Cyber Shield is an Android-based application designed to help users identify potential digital scams. Users can enter or share URLs, scan QR codes, and submit SMS messages or emails for analysis. The Flask backend uses Machine Learning and rule-based checks to classify the submitted content.

The application displays the threat category, confidence score, possible warning signs, and security advice. It also provides user authentication, scan history, profile management, and a Cyber Awareness Challenge. The main objective is to improve online safety and help users make informed decisions before interacting with suspicious content.

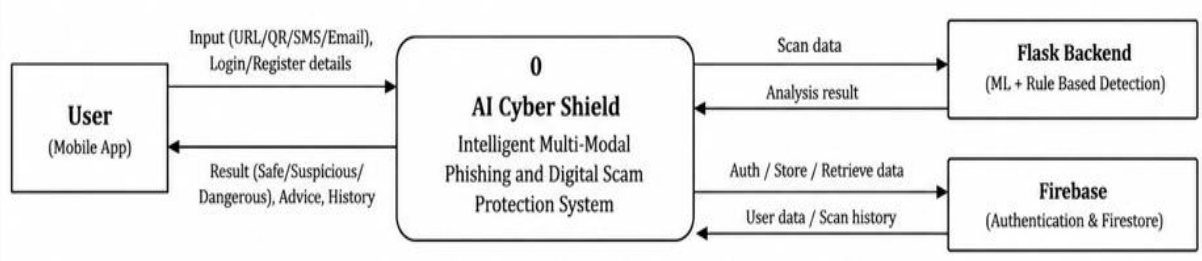
### 3.6 Conceptual Models

Conceptual models represent the working and structure of the system.

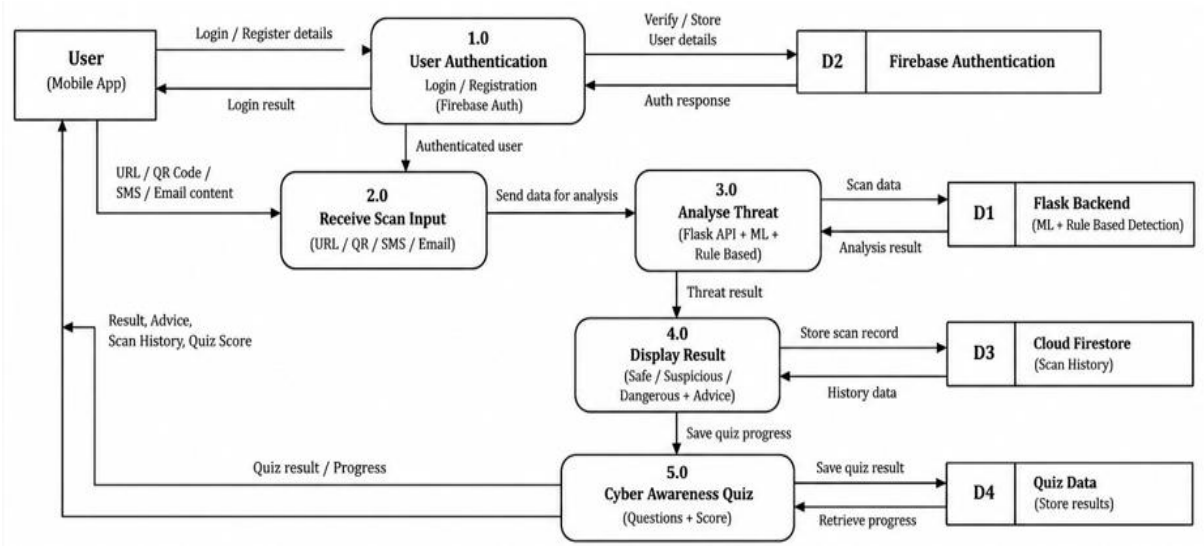
### A: Data Flow Diagram (DFD)

A Data Flow Diagram shows how information moves between the user, the application, the backend and the database.

- Level 0: Shows the complete AI Cyber Shield system as a single process and its interaction with the user, Flask/ML backend and Firebase.

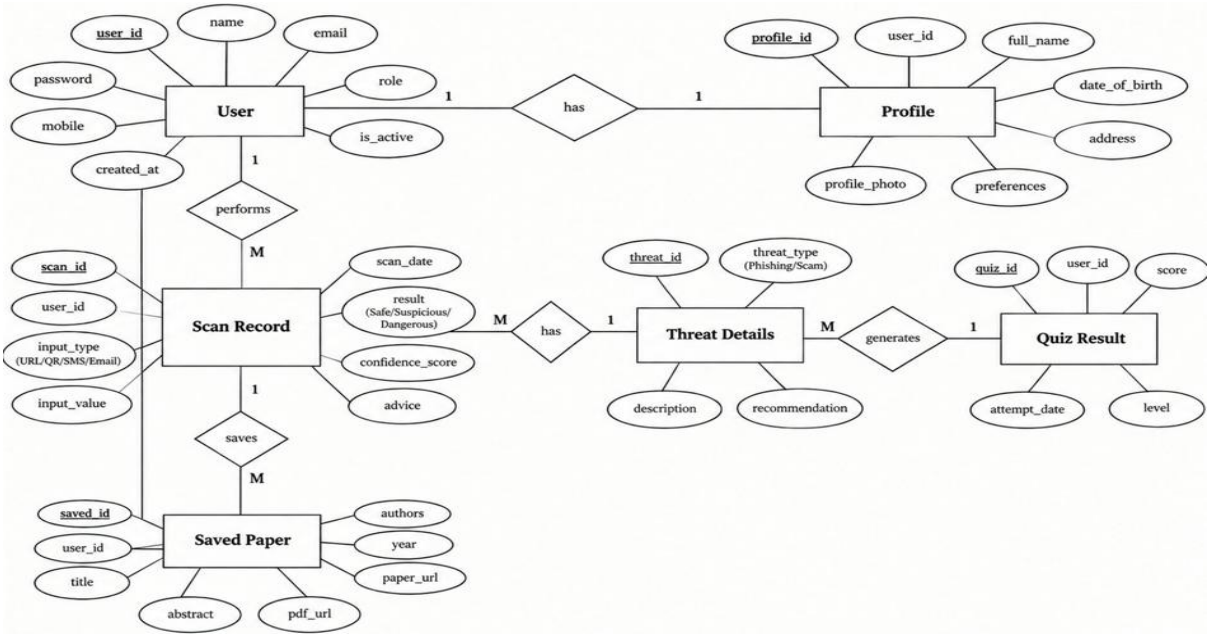



- Level 1: Explains the internal processes, such as user authentication, receiving a URL/QR/SMS/email input, analysing threats, displaying results and storing scan history.



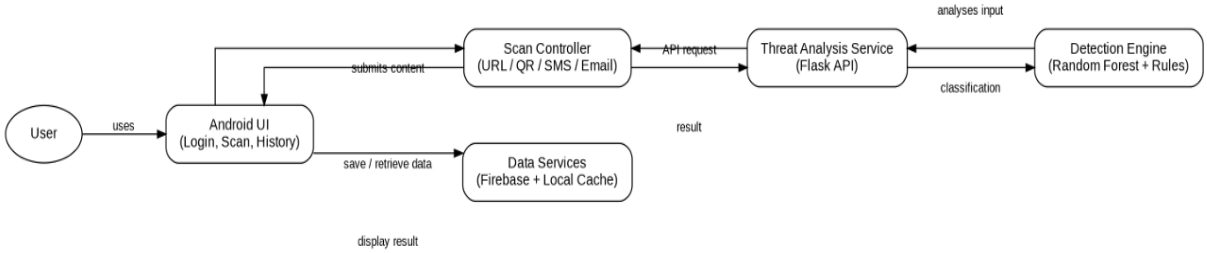
## B: Entity Relationship (ER) Diagram

The ER Diagram represents the data entities and their relationships in the system. It includes User, Profile, Scan Record and Quiz Result. It explains how user information, scan details, threat results and quiz scores are organised and related.



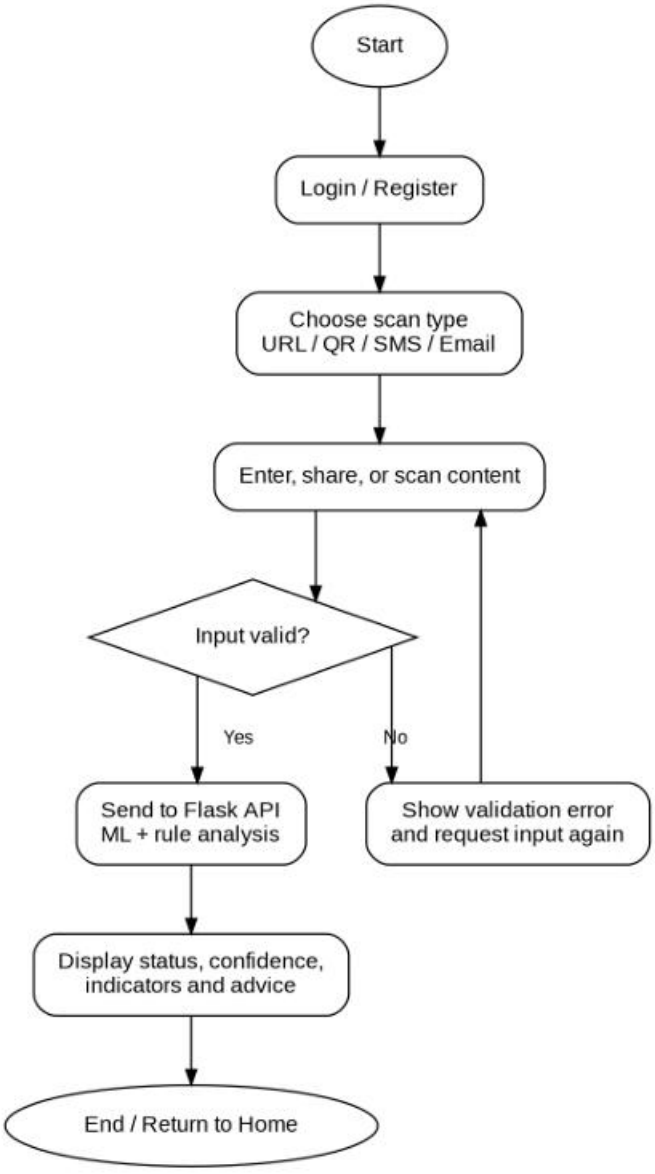

## C: Object-Oriented Diagram

The Object-Oriented Diagram shows the main objects involved in the application and how they interact. It includes the user interface, scan controller, threat analysis service, detection engine and data services. It explains how different objects work together to detect phishing and digital scams.



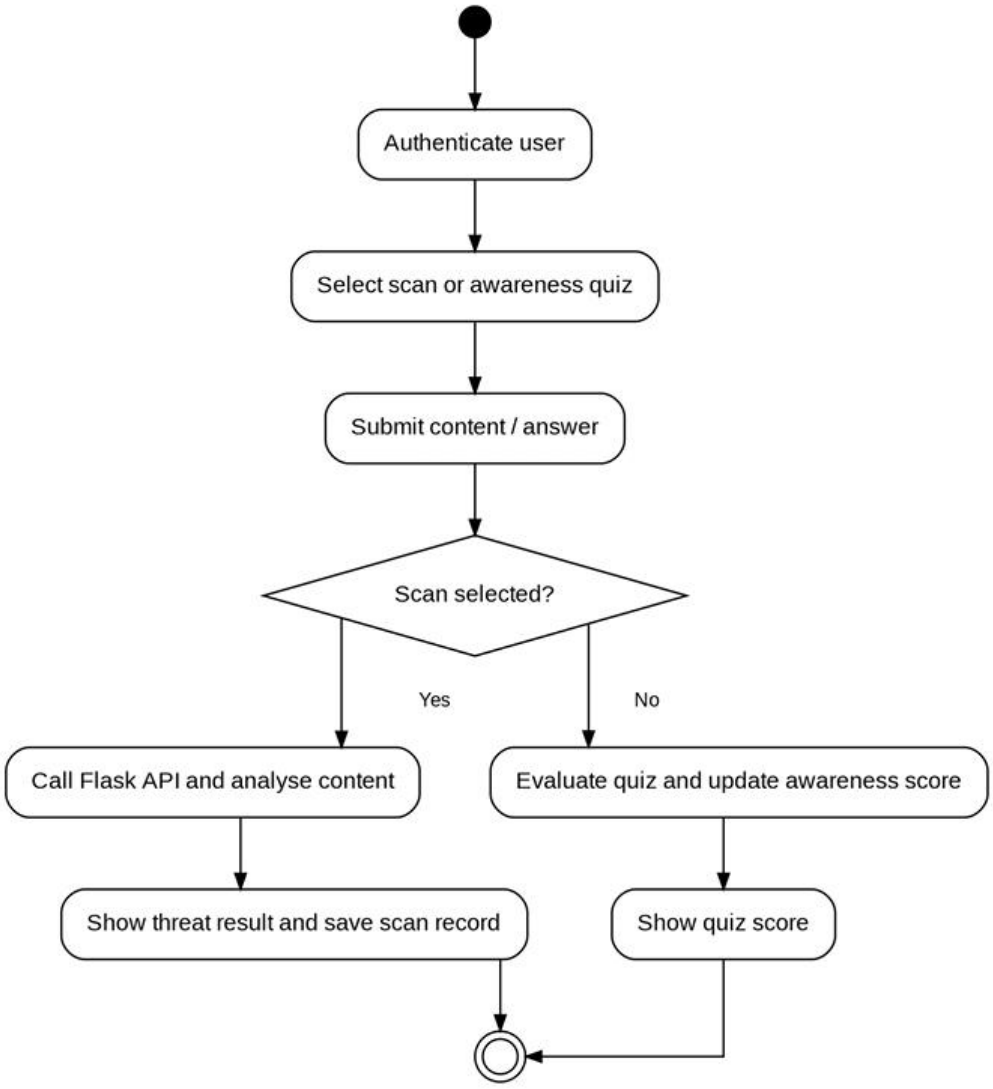
**D: System Flowchart**

The System Flowchart represents the step-by-step working of AI Cyber Shield. The process begins with login or registration, followed by selecting a scanning option, submitting the input, analysing the threat and displaying the result. If the input is invalid, the system displays an error message. Dangerous links are handled through safety warnings.



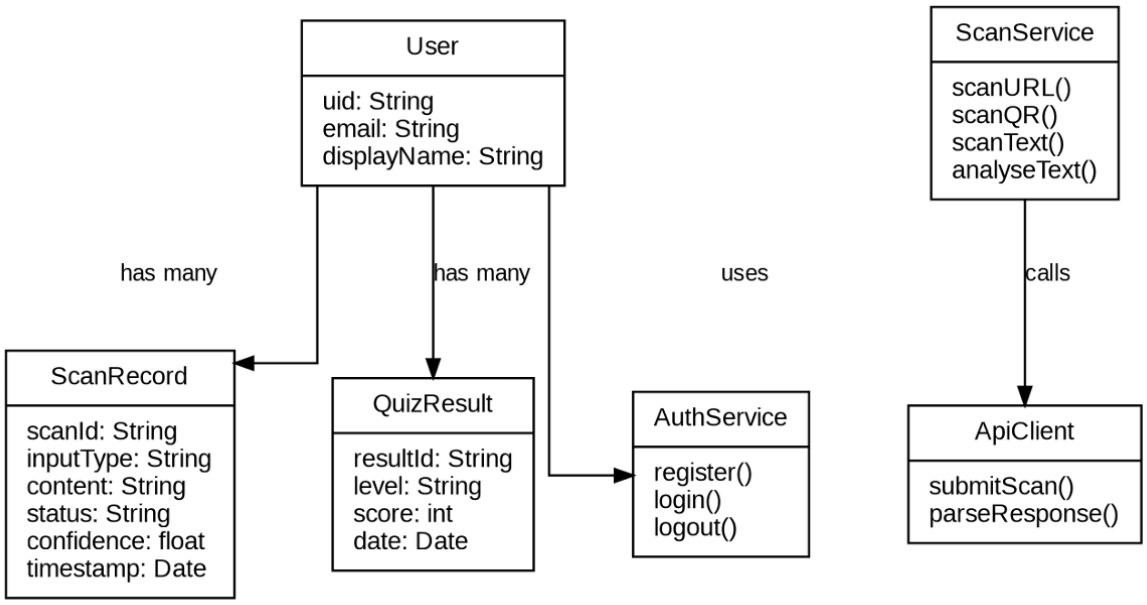
**E: Activity Diagram**

The Activity Diagram describes the activities performed by the user and the system. It shows login, selecting URL/QR/SMS/email scanning or the awareness quiz, submitting input, processing information and displaying the final result or quiz score.



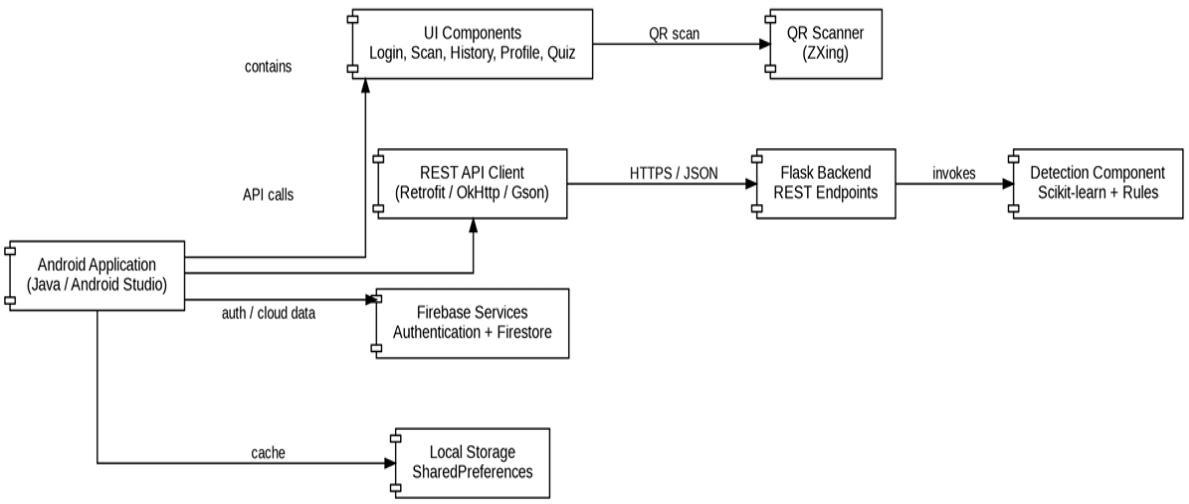
## F: Class Diagram

The Class Diagram represents the main classes and their relationships. It may include User, ScanRecord, QuizResult, AuthService, ScanService and ApiClient. It explains the responsibilities of each class and how the classes support authentication, threat scanning, API communication and record management.



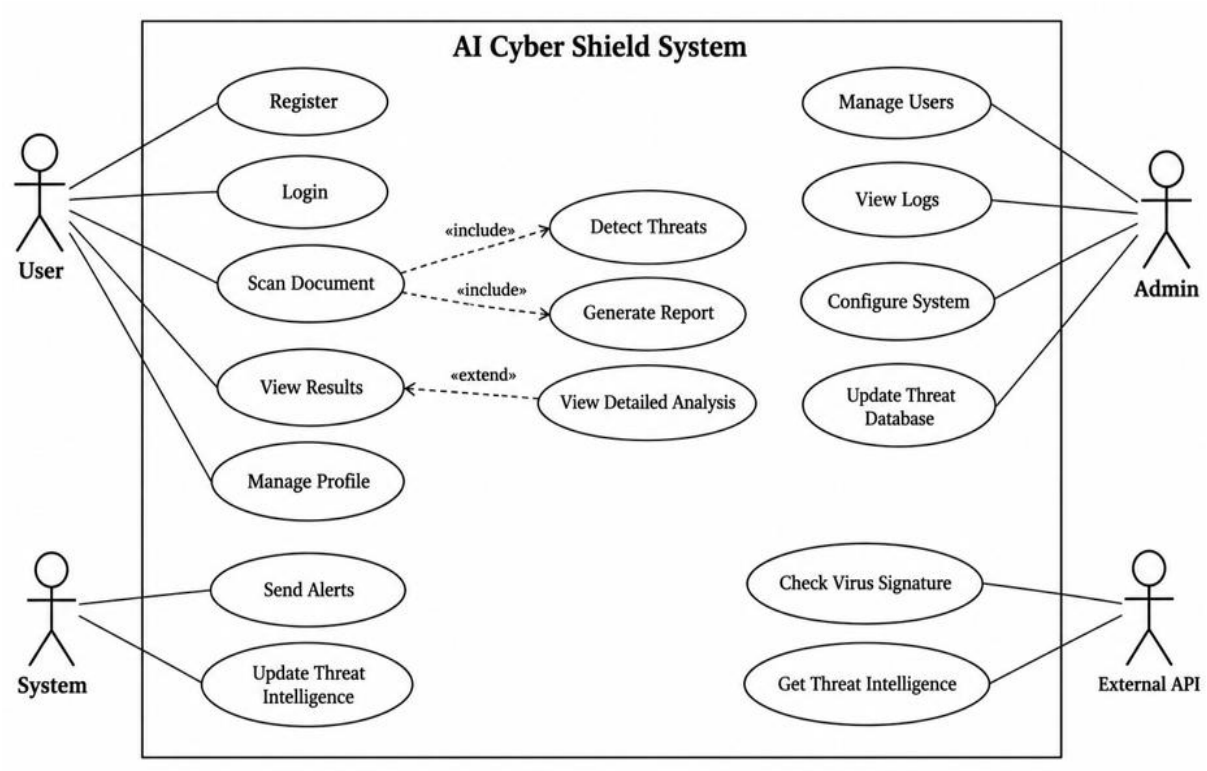
## G: Component Diagram

The Component Diagram shows the major software components and their connections. These include the Android application, REST API client, Flask backend, machine learning and rule-based detection engine, Firebase Authentication, Cloud Firestore and local storage. It explains how these components work together to provide the application's functionality.



## H: Use Case Diagram

The Use Case Diagram represents the interactions between the user and the system. It includes registration, login, scanning URLs and QR codes, checking SMS and email content, viewing threat results, checking scan history, taking awareness quizzes and viewing the profile or awareness score.



# CHAPTER 4:- SYSTEM DESIGN

## 4.1 Basic Modules

To make AI Cyber Shield easier to develop and manage, the system is divided into modules, each handling a specific task. These modules work together to detect phishing links, malicious QR codes, fraudulent SMS messages, and suspicious emails.

1. User Authentication Module: Manages user registration, login, and sign out using Firebase Authentication. User profile information is stored in Cloud Firestore.
2. URL and QR Scanning Module: Accepts a URL or decodes a QR code using ZXing. The extracted URL is checked by the backend before the user opens it.
3. SMS and Email Analysis Module: Analyses message text, sender information, suspicious wording, embedded links, and common social-engineering signals.
4. Threat Detection Module: Uses TF-IDF, Random Forest, and rule-based checks to classify content as SAFE, SUSPICIOUS, or DANGEROUS and returns confidence and indicators.
5. Scan History Module: Saves scan details in Firestore and maintains a local cache for viewing recent scans when the network is unavailable.
6. Cyber Awareness Module: Provides Beginner, Intermediate, and Advanced quizzes and calculates the user's awareness score.
7. Safety and User Interface Module: Displays results and warnings through the Android interface and blocks direct opening of links marked DANGEROUS.

### 4.2 Data Design

Data design explains how the system stores, organises, and manages user information and scan results. AI Cyber Shield uses Firebase Authentication to manage user identity and Cloud Firestore to store user profiles and scan records. Local storage may be used to cache scan history for access when the internet connection is unavailable.

Each scan record contains details such as the input type, scanned content, threat status, confidence value, reason, indicators, and timestamp. This organised structure makes it easier to retrieve and display previous scan results.

### 4.2.1 Schema Design

The database schema defines how user information and scan records are organised in Firestore.

The proposed Firestore structure is:

| Document/Field | Data Type | Description |
|---|---|---|
| users/{userId} | Document | User profile identified by Firebase UID. |
| name, email | String | Basic user information. |
| createdAt | String | Account creation date and time. |
| awarenessScore | Integer (0–100) | Cyber-awareness quiz score. |
| challengesCompleted | Integer | Number of completed quizzes. |
| scans/{scanId} | Subcollection document | One record for each completed scan. |
| Type | String / Enum | LINK, QR, SMS, or EMAIL. |
| Target | String | URL or text content analysed. |
| Status | String / Enum | SAFE, SUSPICIOUS, or DANGEROUS. |
| Confidence | Integer (0–100) | Confidence percentage returned by analysis. |
| reason, indicators | String, Array | Explanation and threat signals. |
| timestamp | string | Date and time of the scan. |

### 4.2.2 Data Integrity and Constraints

Data integrity ensures that the information stored in the system is accurate, complete, and associated with the correct user. The following constraints should be applied:

1. User authentication: Only authenticated users should access protected account features.
2. User-specific records: Each scan should be stored under the correct user's ID.
3. Valid input type: The supported scan types are LINK, QR, SMS, and EMAIL.
4. Valid threat status: The classification should be SAFE, SUSPICIOUS, or DANGEROUS.
5. Confidence value: If the backend returns confidence as a percentage, it should be within the range of 0 to 100.
6. Required fields: Important fields such as type, target, status, and timestamp should be validated before saving.
7. Input validation: Empty or invalid input should be rejected with an appropriate message.

8. Access control: Firestore Security Rules should prevent one user from accessing another user's private scan records.

These constraints help maintain reliable scan history and reduce incorrect or unauthorised data access.

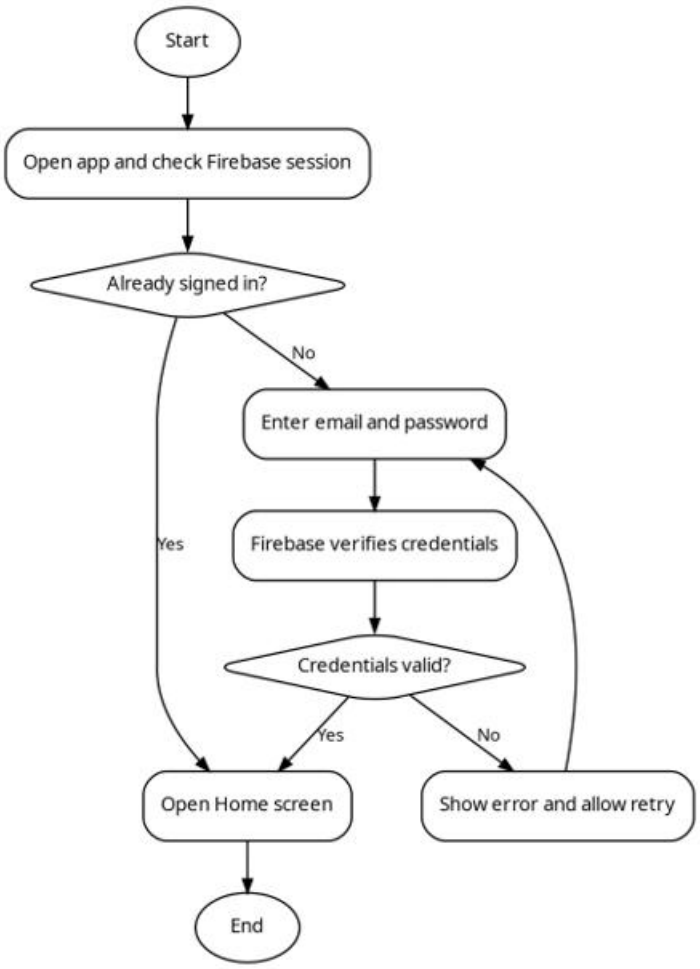
### 4.3 Procedural Design

Procedural design describes the sequence of steps followed by AI Cyber Shield to analyse suspicious content, display the result, and manage scan records. The process starts when the user enters a URL, scans a QR code, or provides SMS or email content. The system validates the input, sends supported content to the backend, performs threat analysis, and displays the result.

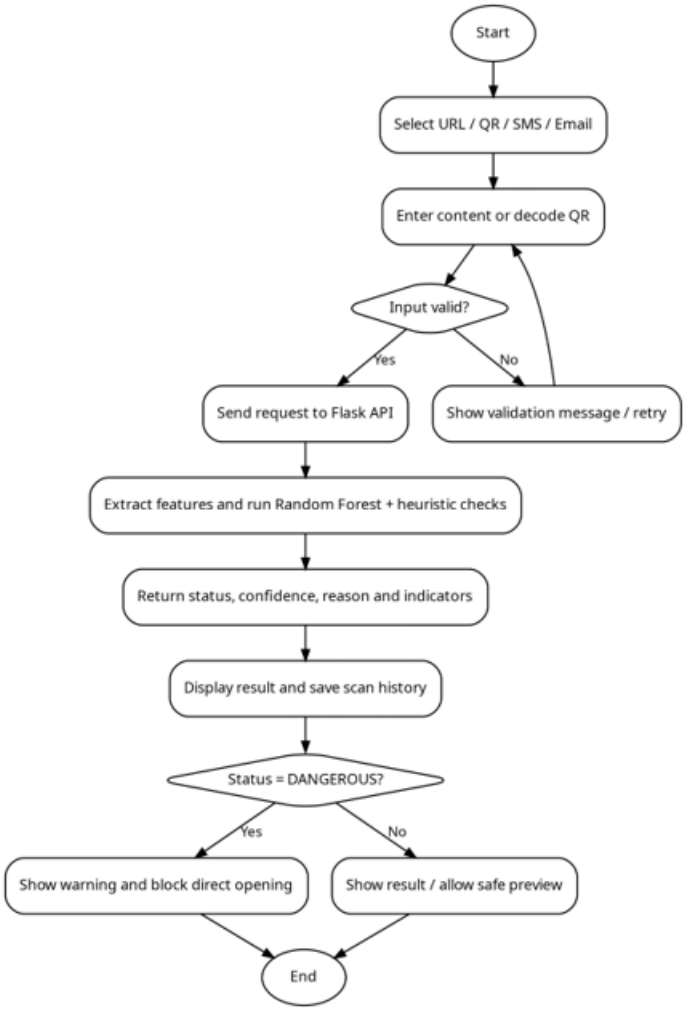
#### 4.3.1 Logic Diagrams

The following flowcharts show the sequence of operations and decisions in the main AI Cyber Shield processes.

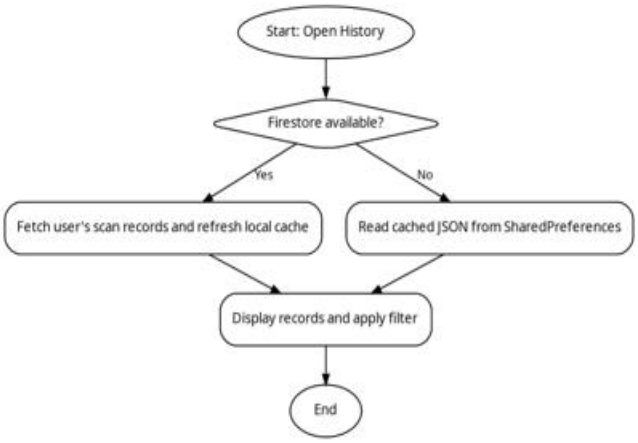
**Logic Flow:**



**Threat Scanning Flow:**




**Scan History Flow:**



### 4.3.2 Data Structure

Data structures are used to store and organise the information processed by the application. AI Cyber Shield uses strings for textual information, integers for numerical values, lists for detected indicators, JSON objects for API communication, and Firestore documents for storing user and scan data.

| Data Item | Data Type | Constraint/Description |
|---|---|---|
| userId | String | Unique Firebase UID. |
| scanId | String | Unique scan identifier. |
| Type | String / Enum | LINK, QR, SMS, EMAIL. |
| Target | String | Scanned URL or message text. |
| Status | String / Enum | SAFE, SUSPICIOUS, DANGEROUS. |
| Confidence | Integer | Value from 0 to 100. |
| Indicators | Array of Strings | Threat indicators returned by analysis. |
| Timestamp | String | Scan date and time. |
| Local cache | JSON | Recent scan records stored locally |

### 4.3.3 Algorithms Design

Algorithm Design explains the steps used to process input and produce an output. AI Cyber Shield uses separate input-processing procedures for URL, QR, SMS, and email scans, while presenting their results through a common threat-result interface.

**Algorithm 1: Multi-Model phishing and Scam Detection**

**Input:** URL, decoded QR payload, SMS text, or email sender/subject/body.

**Output:** Threat status, confidence, reason, indicators, and security advice.

1. Accept content from the selected scanner screen.
2. Decode QR codes using ZXing to extract the payload.
3. Validate the input and select the appropriate Flask endpoint: /predict-url, /predict-qr, /predict-sms, or /predict-email.
4. Send the request from Android to Flask using Retrofit and REST/JSON.

5. Extract URL features such as length, domain, protocol, suspicious TLDs, and brand-spoofing clues.
6. Analyse SMS and email text for suspicious wording, urgency, financial triggers, sender mismatches, and embedded URLs.
7. Apply TF-IDF vectorization, the trained Random Forest classifier, and rule-based checks.
8. Classify the content as SAFE, SUSPICIOUS, or DANGEROUS with confidence and indicators.
9. Display the result and save the scan to the user's history.
10. If DANGEROUS, block direct link opening and display a safety warning.

**Algorithm 2: Scan History Strorage and Retrieval**

**Input:** Completed scan record and current user's authentication state.

**Output:** Stored scan record and scan-history entries.

1. Create a scan record with ID, type, target, status, confidence, reason, indicators, and timestamp.
2. Save the record to users/{userId}/scans/{scanId} in Firestore when cloud access is available.
3. Update the local SharedPreferences cache using Gson and retain the latest 50 scans.
4. When History opens, fetch scan records for the signed-in user from Firestore.
5. If retrieval succeeds, display the records and refresh the local cache.
6. If cloud access fails, display available records from the local JSON cache.
7. Apply the selected All, Safe, Suspicious, or Dangerous filter to the displayed records.

**Algorithm 3: Cyber Awareness Score Calculation**

**Input:** Number of questions attempted and correct answers.

**Output:** Awareness score percentage and updated profile values.

1. Count the total questions and correct answers.
2. If the total is greater than zero, calculate the score as (correct answers ÷ total questions) × 100.
3. Display the percentage, answer feedback, and corresponding rank badge.
4. Update awarenessScore and challengesCompleted in the user's Firestore profile when cloud access is available.

### 4.4 User Interface Design

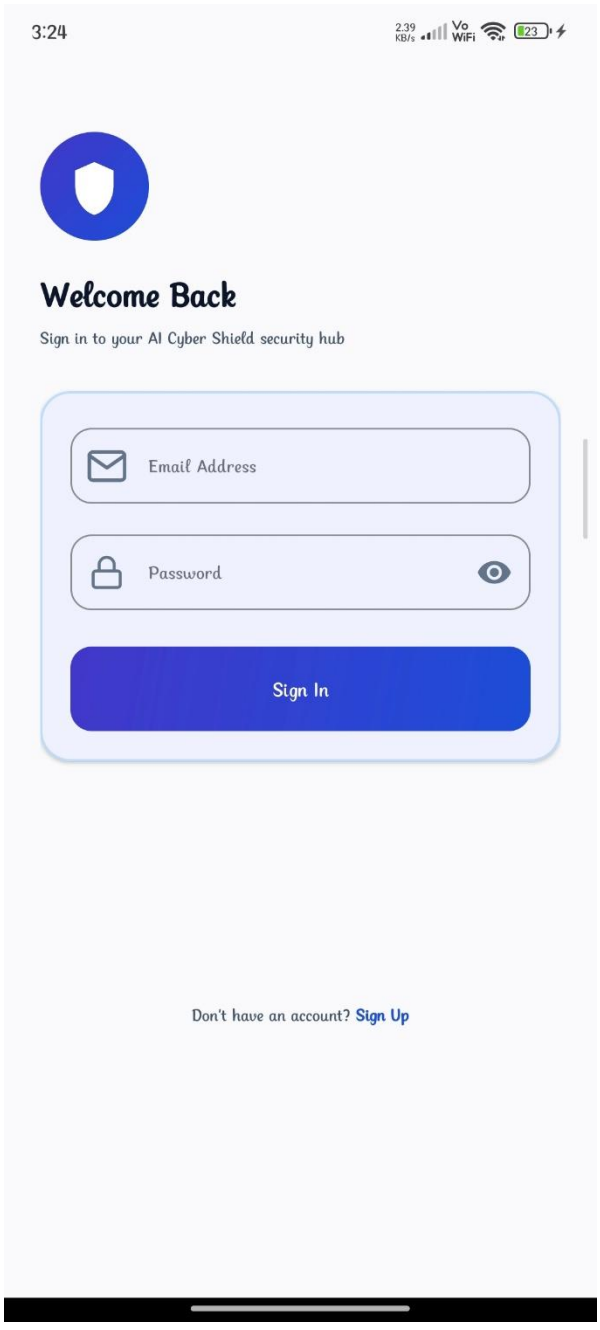
The user interface is designed to be simple, clear, and easy to understand. It allows users to submit suspicious content and understand the security result without requiring advanced cybersecurity knowledge.

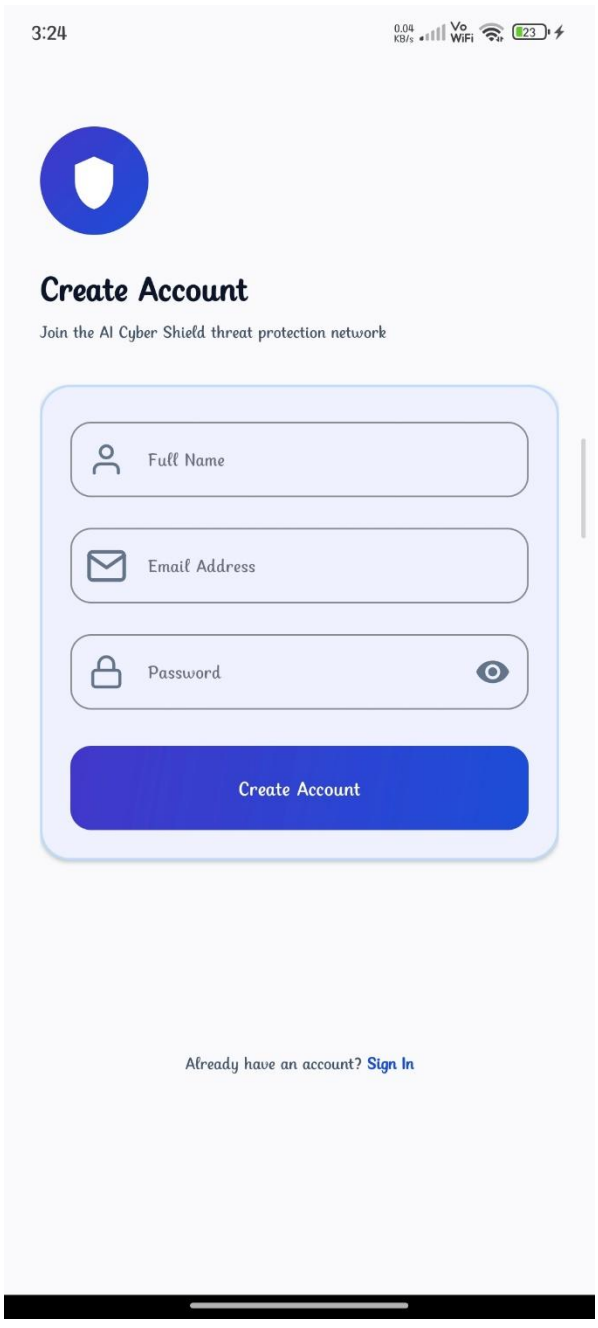
The application uses Android Studio, Java, and Material Design components. A blue-and-white colour scheme provides a consistent appearance. Different result colours can help users recognise the threat classification: green for SAFE, amber for SUSPICIOUS, and red for DANGEROUS.

The main screens are described below.

**Login/Registration:**

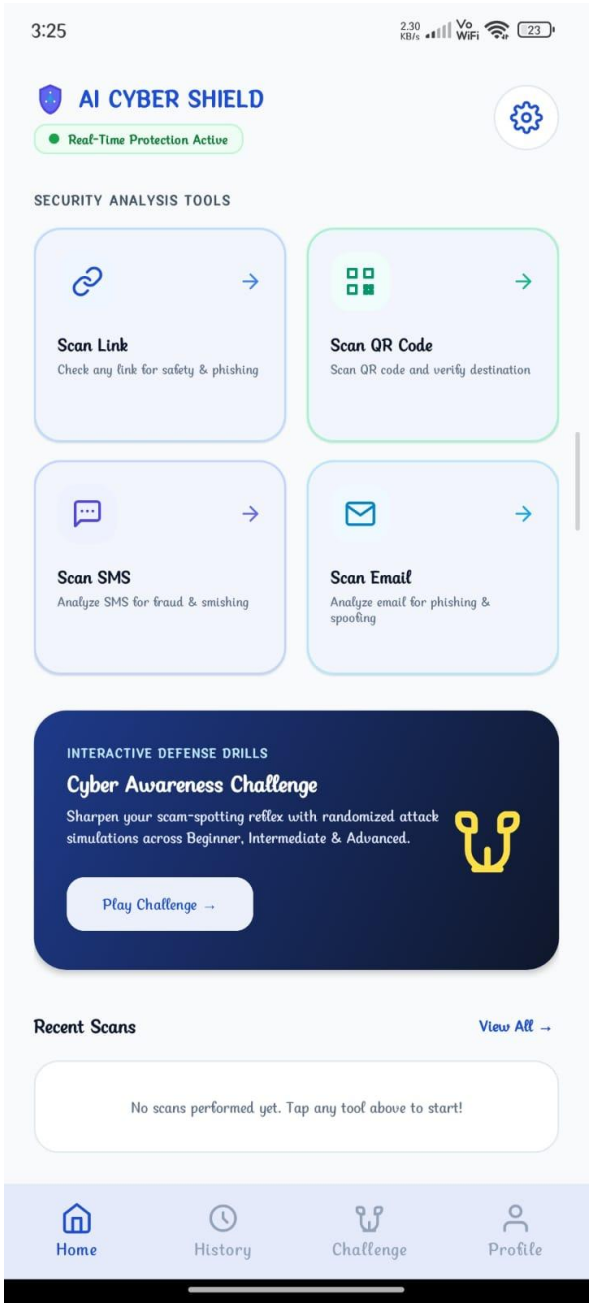
**User authentication and account creation.**

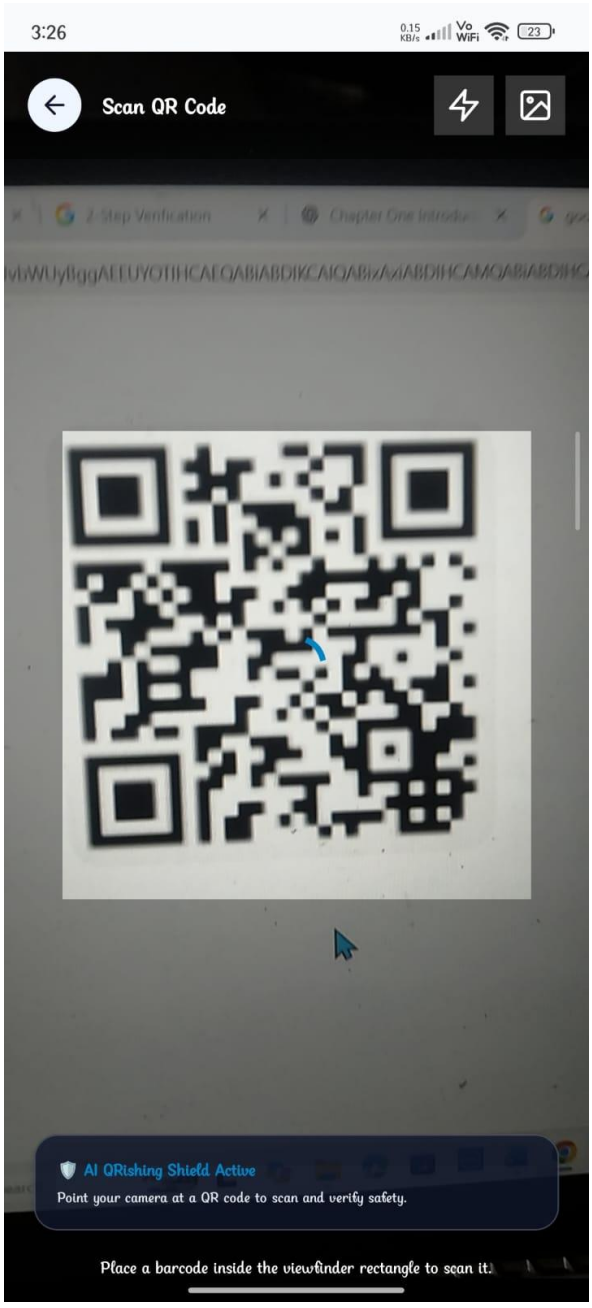


**Home:  Access to URL, QR, SMS, and email scanning.**

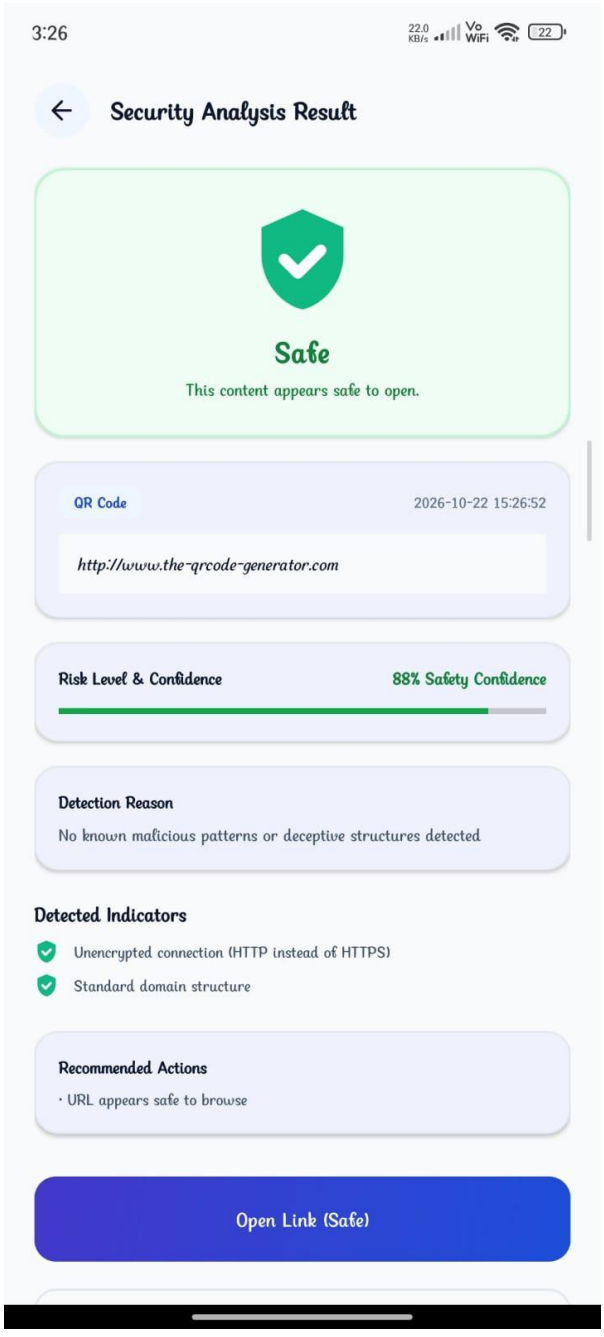
**Scanner Screen: Enter or capture content for analysis.**

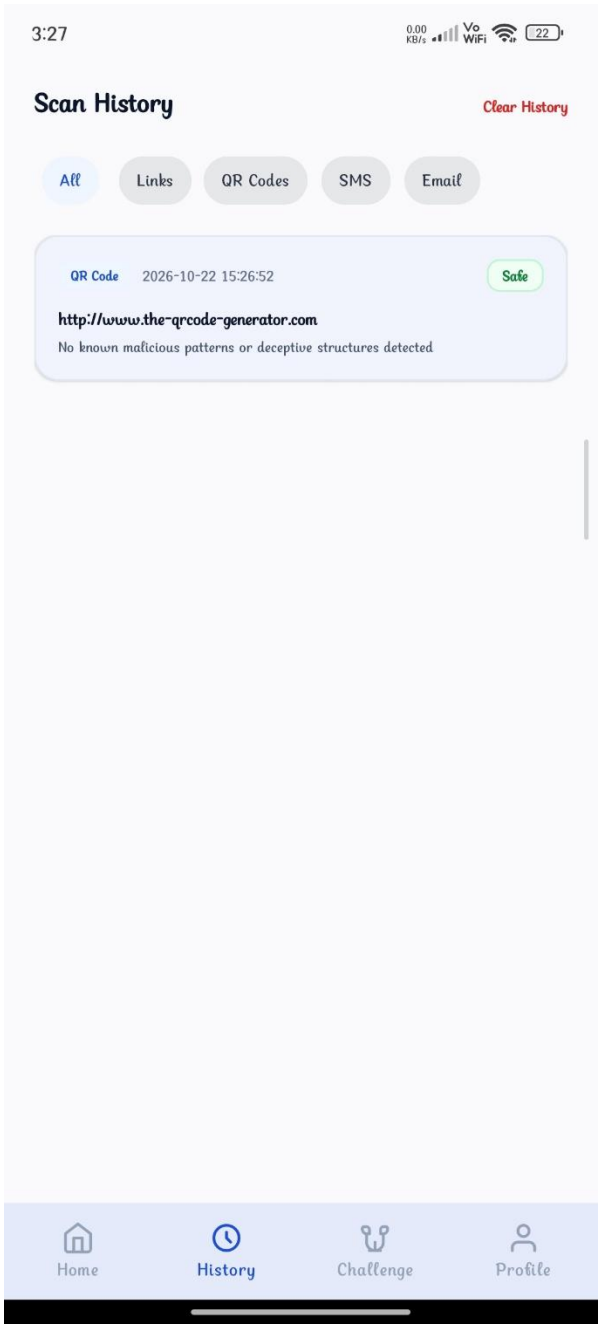


**Threat Result: Shows status, confidence, reason, and advice.**

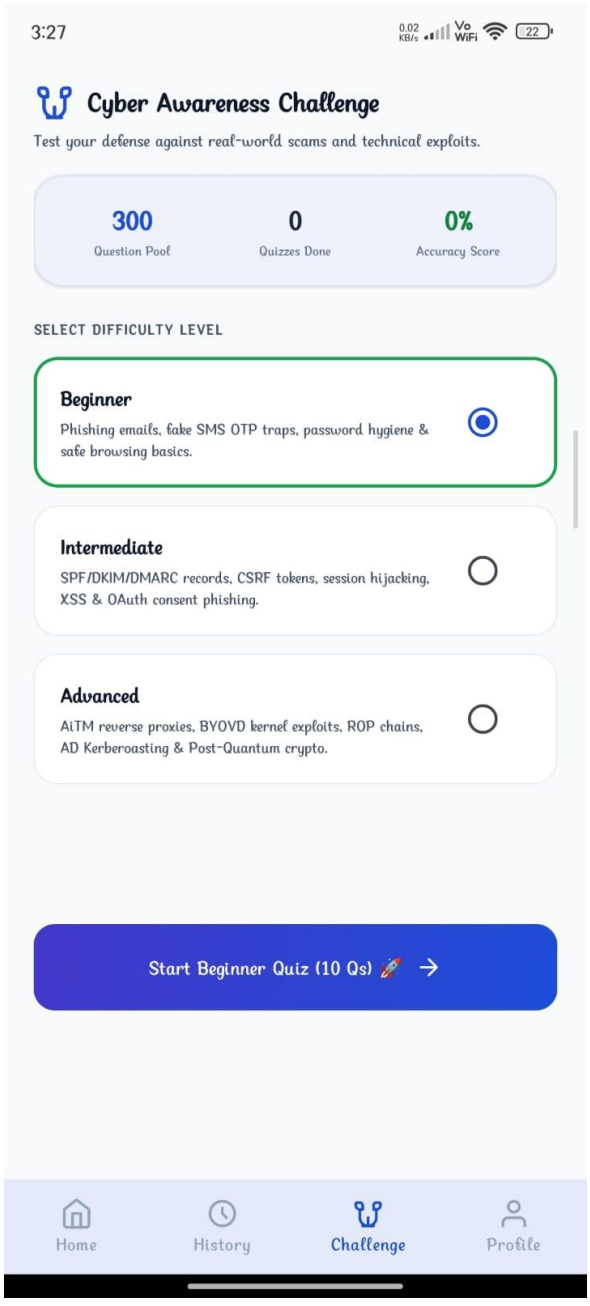
**History: Displays saved scans and verdict filters.**

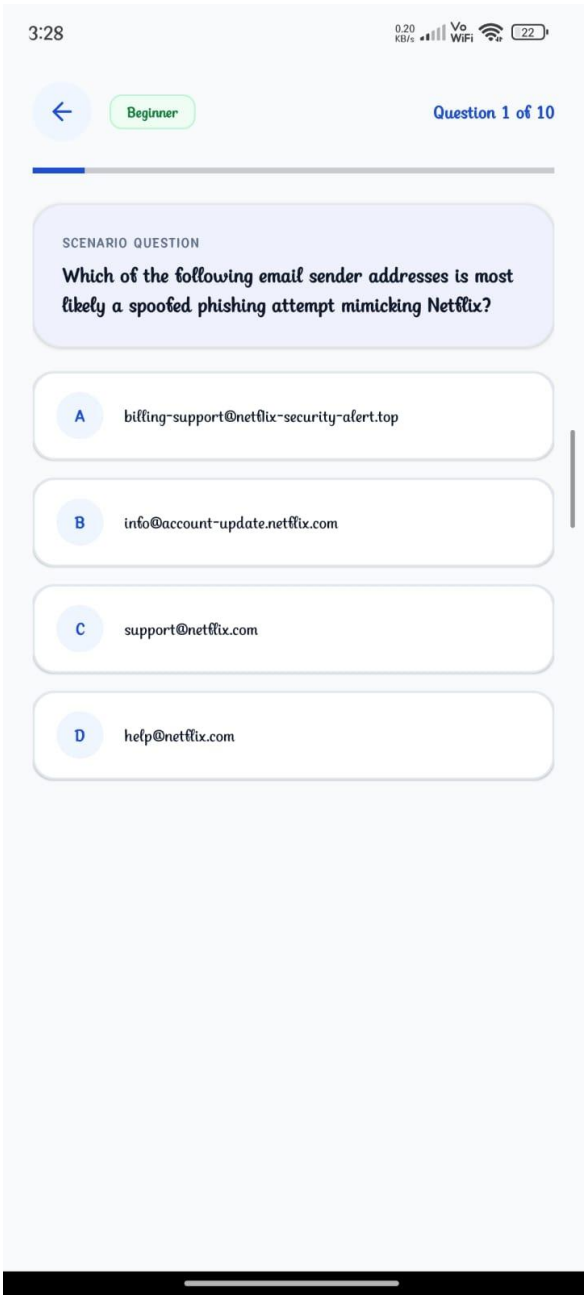


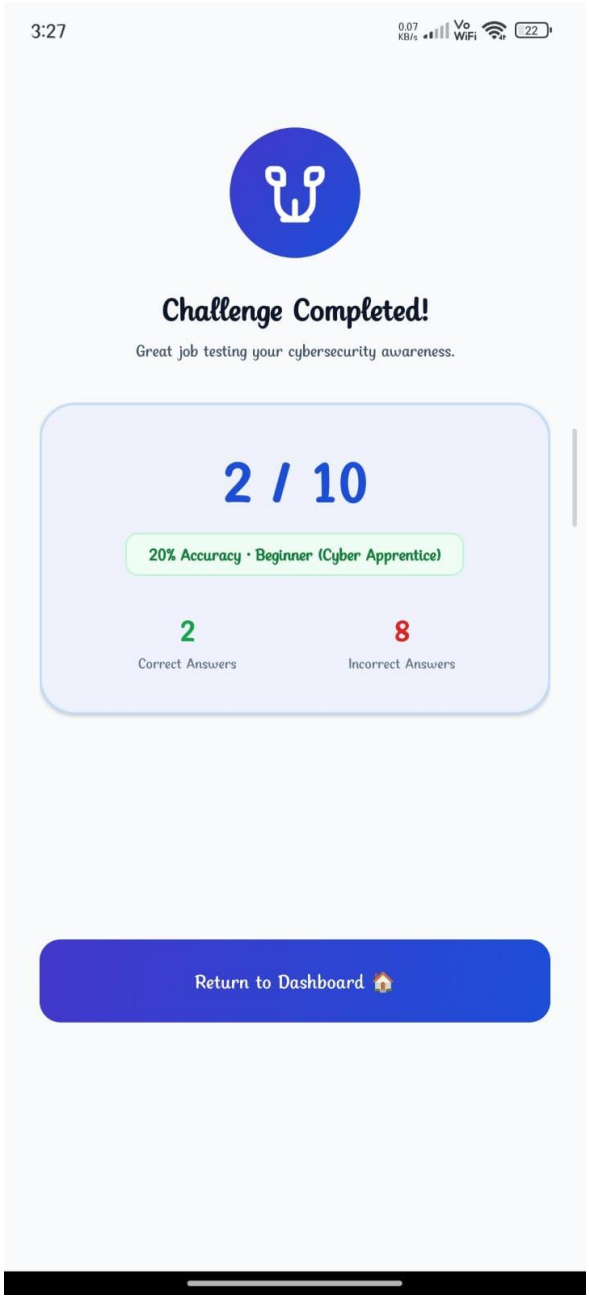



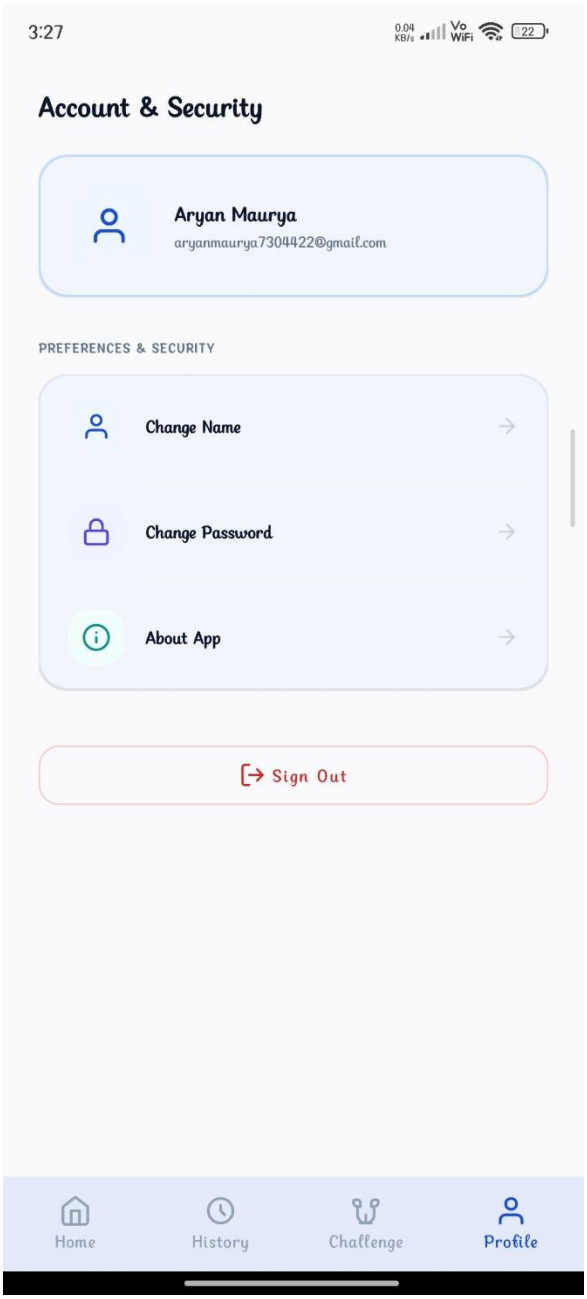
**Challenge: Provides quiz questions and score feedback.**

**Profile: Displays user and awareness information.**




3:27
Challenge Completed!
Great job testing your cybersecurity awareness.
2 / 10
20% Accuracy · Beginner (Cyber Apprentice)
2
Correct Answers
8
Incorrect Answers
Return to Dashboard


3:27
Account & Security
Aryan Maurya
aryanmaurya7304422@gmail.com
PREFERENCES & SECURITY
Change Name
Change Password
About App
Sign Out
Home
History
Challenge
Profile

### 4.5 Security Issues

Security is important in AI Cyber Shield because it analyses URLs, QR codes, SMS messages, and email content. The system must protect user data and handle suspicious content safely.

1. User Authentication: Firebase Authentication verifies user identities and protects account access.

2. Database Access Control: Firestore Security Rules should prevent users from accessing other users' private records.

3. Input Validation: Empty, invalid, or unsupported inputs should be rejected safely.

4. Dangerous Link Protection: Dangerous links should not open directly, and users should receive a warning.

5. Secure Communication: HTTPS should be used to protect data transmitted between the application and backend.

6. Privacy Protection: Passwords and OTPs should not be stored in scan history or application logs.

7. Camera Permission: Camera access should be requested only when required for QR scanning.

8. Safe WebView Handling: If WebView is used, unsafe navigation and unnecessary JavaScript access should be restricted.

9. Error Handling: Backend or network failures should display an error instead of an unverified SAFE result.

10. Detection Limitations: A SAFE result does not guarantee complete safety. Users should remain cautious with unknown links and messages.

These measures help improve the security, privacy, and reliability of AI Cyber Shield.

**4.6 Test Case Design**

| TC ID | Module | Test Scenario | Expected Result | Status |
|---|---|---|---|---|
| TC_01 | Login | Login with valid email and password | User successfully logs in and navigates to Home Dashboard. | Pass |
| TC_02 | Registration | Create account with valid name, email and password | User account is created successfully. | Pass |
| TC_03 | Home Dashboard | Verify all security tools are displayed | Scan Link, Scan QR Code, Scan SMS and Scan Email options are visible. | Pass |
| TC_04 | Link Scanner | Scan a valid URL | Security analysis result displays the URL's safety status and risk information. | Pass |
| TC_05 | QR Scanner | Scan a QR code | The app reads the QR code and displays its security analysis result. | Pass |
| TC_06 | SMS Scanner | Analyze an SMS message | The app analyzes the message and displays the relevant security result. | Pass |
| TC_07 | Email Scanner | Analyze an email for phishing | The app displays the email security analysis result. | Pass |
| TC_08 | Security Result | Verify scan result details | Safety status, confidence score, detection reason and recommended actions are displayed. | Pass |

| | | | | |
|---|---|---|---|---|
| TC_09 | Scan History | View previous scan records | Previous scans appear with scan type, date, URL or message details and safety status. | Pass |
| TC_10 | Scan History | Filter records by scan category | All, Links, QR Codes, SMS and Email filters display the corresponding records. | Pass |
| TC_11 | Scan History | Clear scan history | Scan history is cleared and the updated history screen is displayed. | Pass |
| TC_12 | Cyber Challenge | Select difficulty level | Beginner, Intermediate and Advanced difficulty options can be selected. | Pass |
| TC_13 | Cyber Challenge | Complete a quiz | Quiz result displays total score, correct answers, incorrect answers and accuracy percentage. | Pass |
| TC_14 | Profile | Change account name or password | Updated account information or password is saved successfully. | Pass |
| TC_15 | Profile | Sign out from the application | User is logged out and redirected to the Login screen. | Pass |

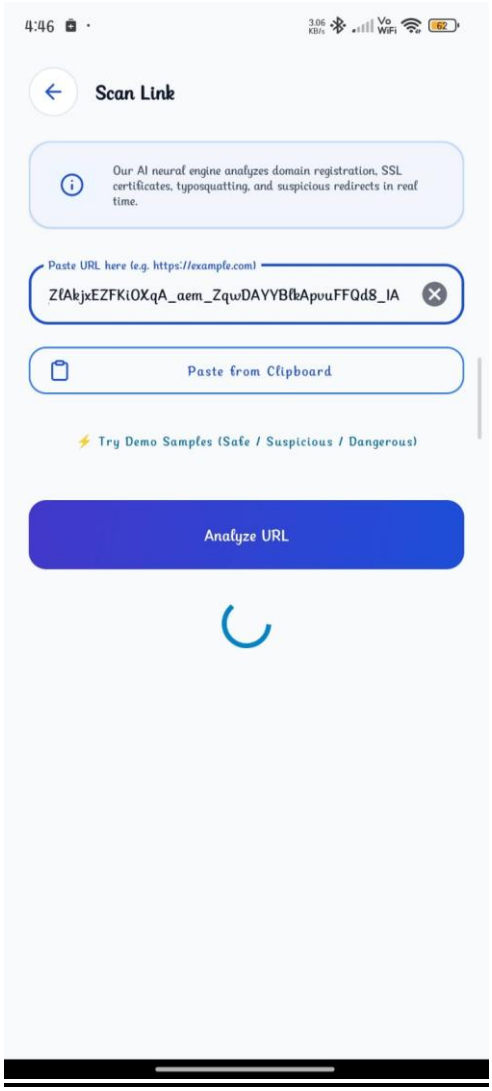
# CHAPTER 5:- RESULT AND DISCUSSION

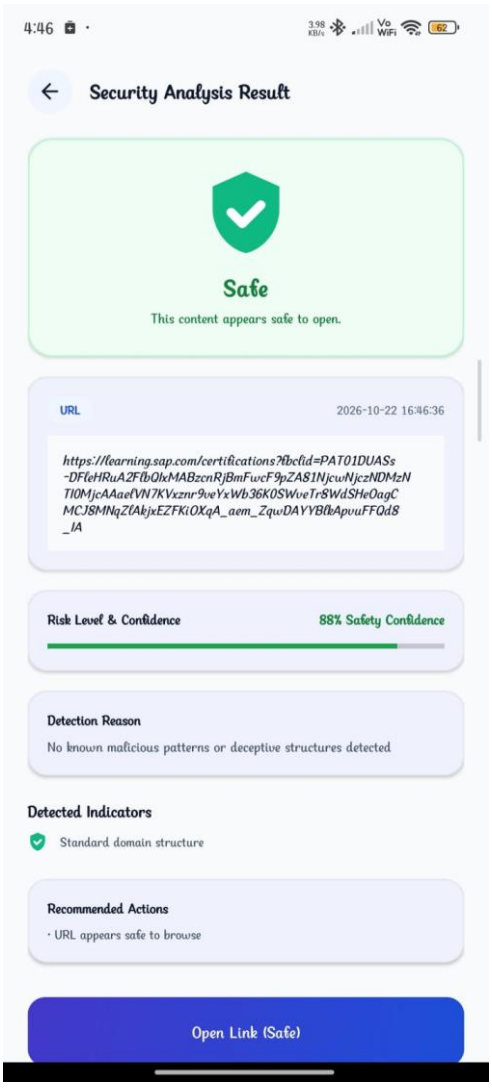
AI Cyber Shield – Intelligent Multi-Modal Phishing and Digital Scam Protection System is developed to help users identify potentially harmful URLs, QR codes, SMS messages, and email content. The system uses a machine-learning model and rule-based checks to classify submitted content as SAFE, SUSPICIOUS, or DANGEROUS.

This chapter presents the main outputs of the system, their descriptions, and a discussion of the results. The user interface screenshots and test case table are already included in Chapter 4, so they are not repeated here.

## 5.1 Main System Results

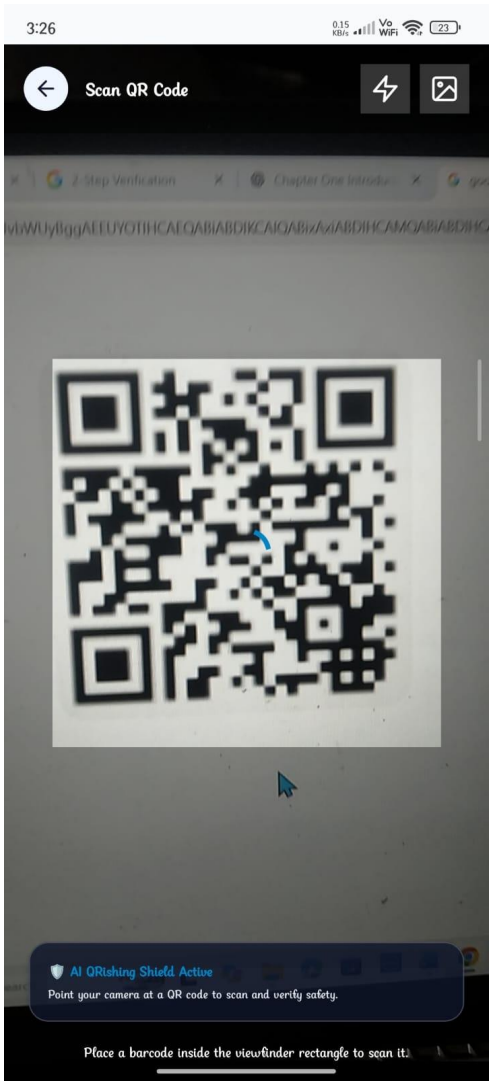
### 5.1.1 URL Scanning Result

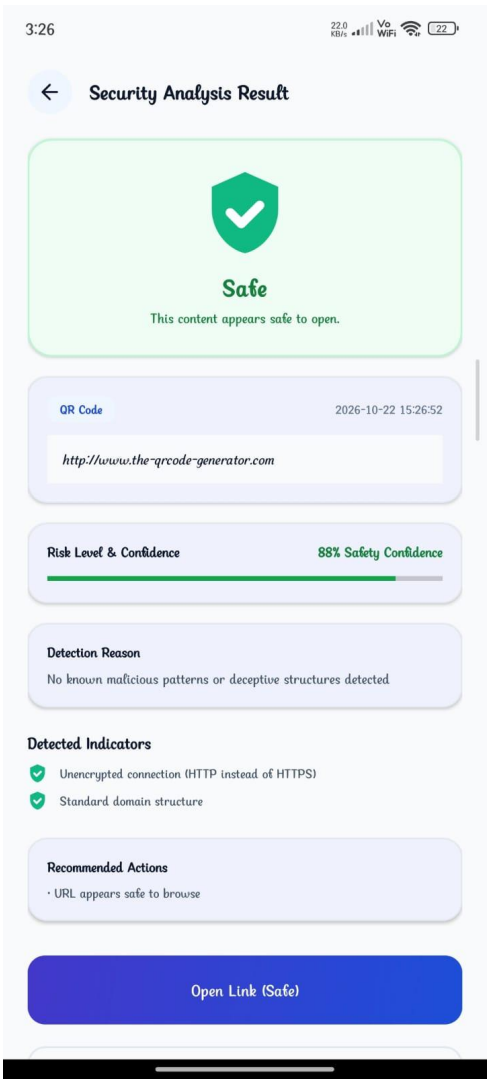


**Description:** The URL scanning feature analyses a submitted website address using the Flask backend. The system checks URL characteristics and applies the implemented detection logic to identify potential threats. The result screen displays the threat status, confidence, detected indicators, and safety advice when available.

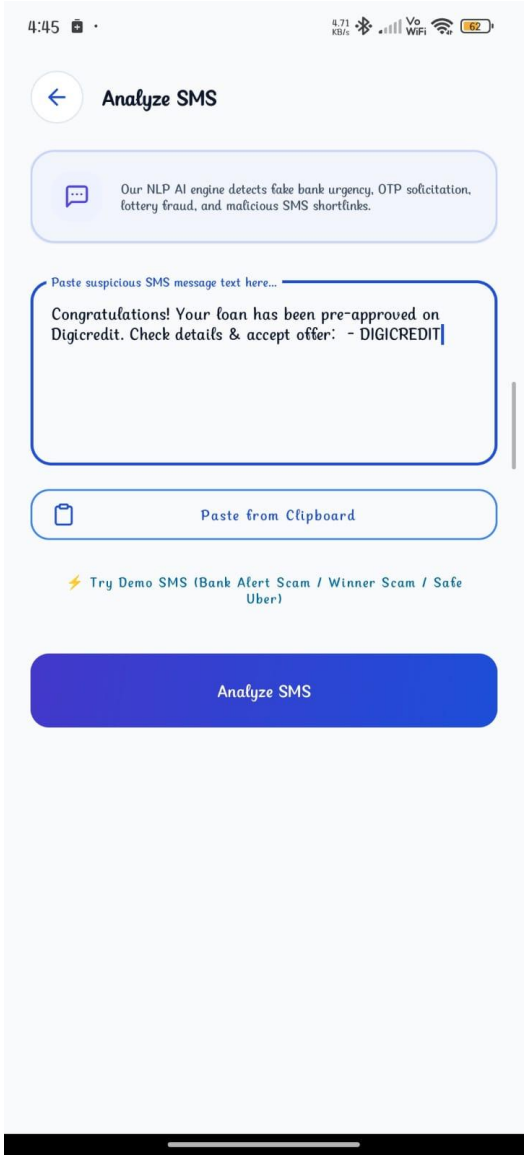
### 5.1.2 QR Code Scanning Result

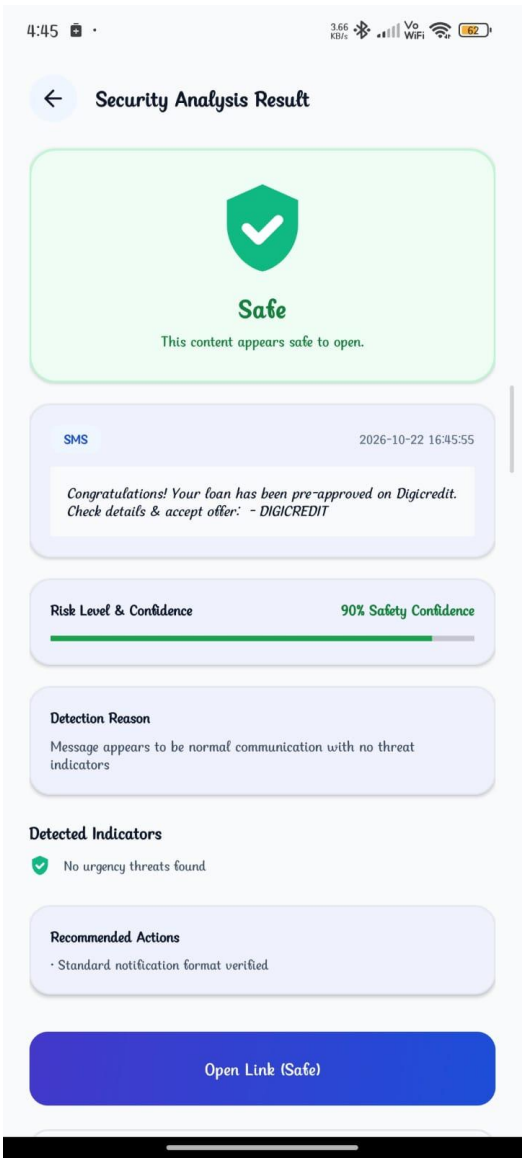



**Description:** The QR scanning feature uses the ZXing library to extract content from a QR code. The extracted content is sent for analysis when supported. The system displays the resulting threat classification to help users identify potentially harmful QR-linked content.

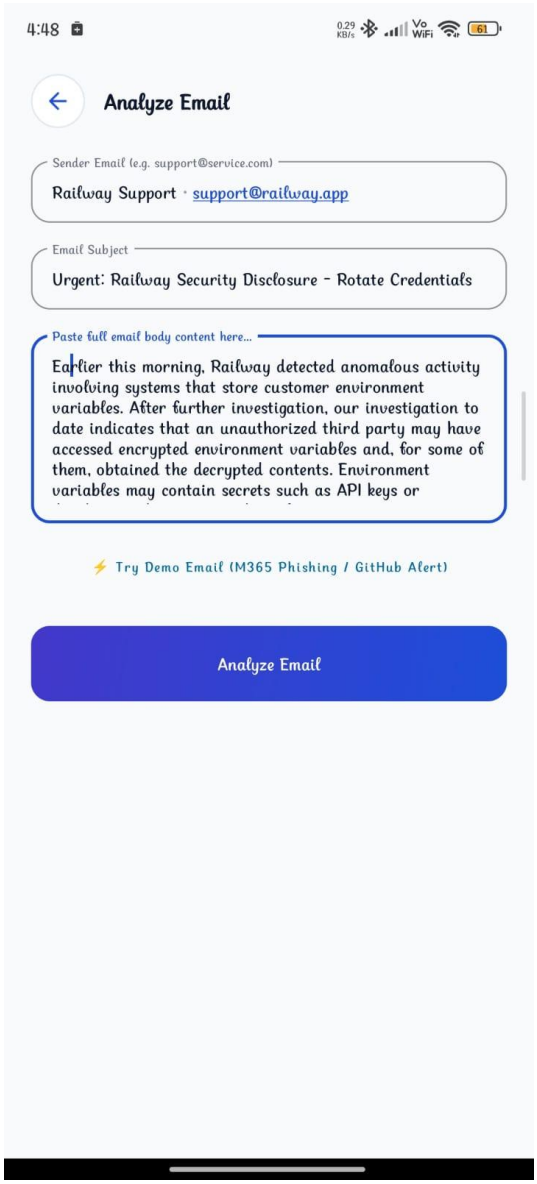
### 5.1.3 SMS Analysis Result

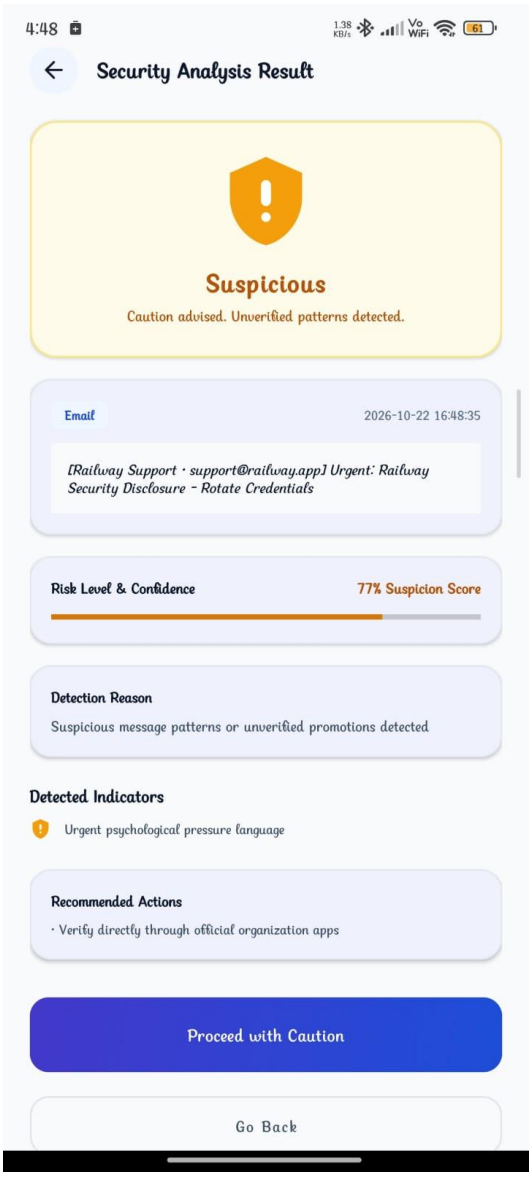



**Description:** The SMS analysis feature checks the submitted text message for suspicious words, urgent requests, financial fraud indicators, and harmful links. The system analyses the message using machine learning and rule-based checks to identify possible scams. The result screen displays the threat status, confidence level, reason, and indicators to help users understand whether the message may be fraudulent.

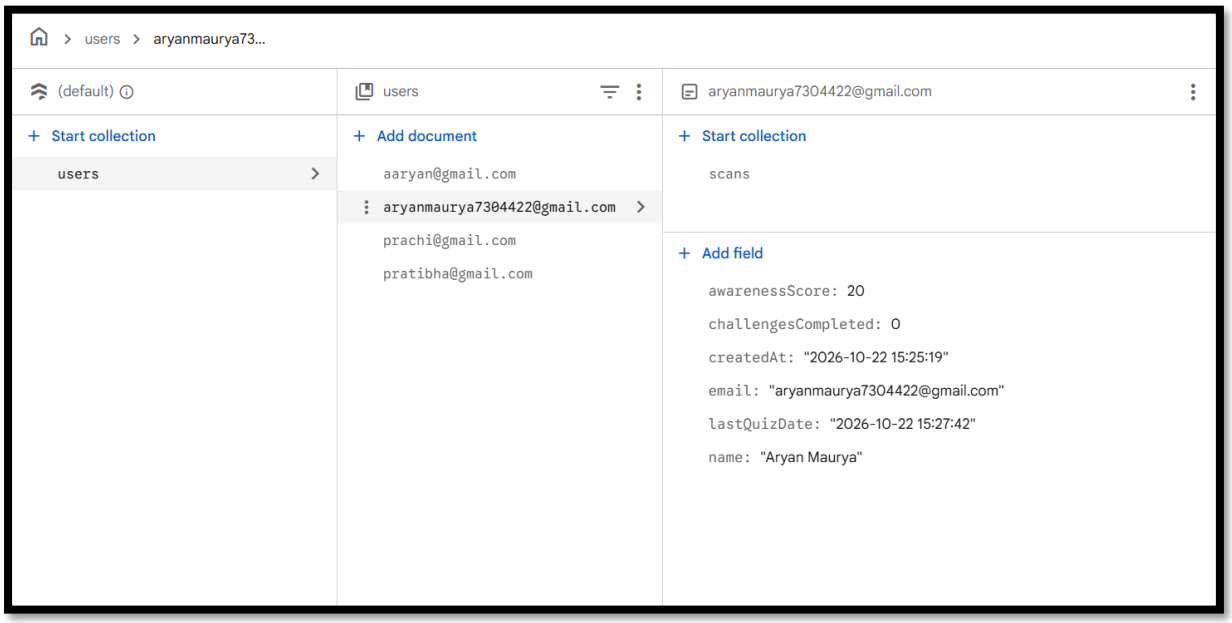
### 5.1.4 Email Analysis Result

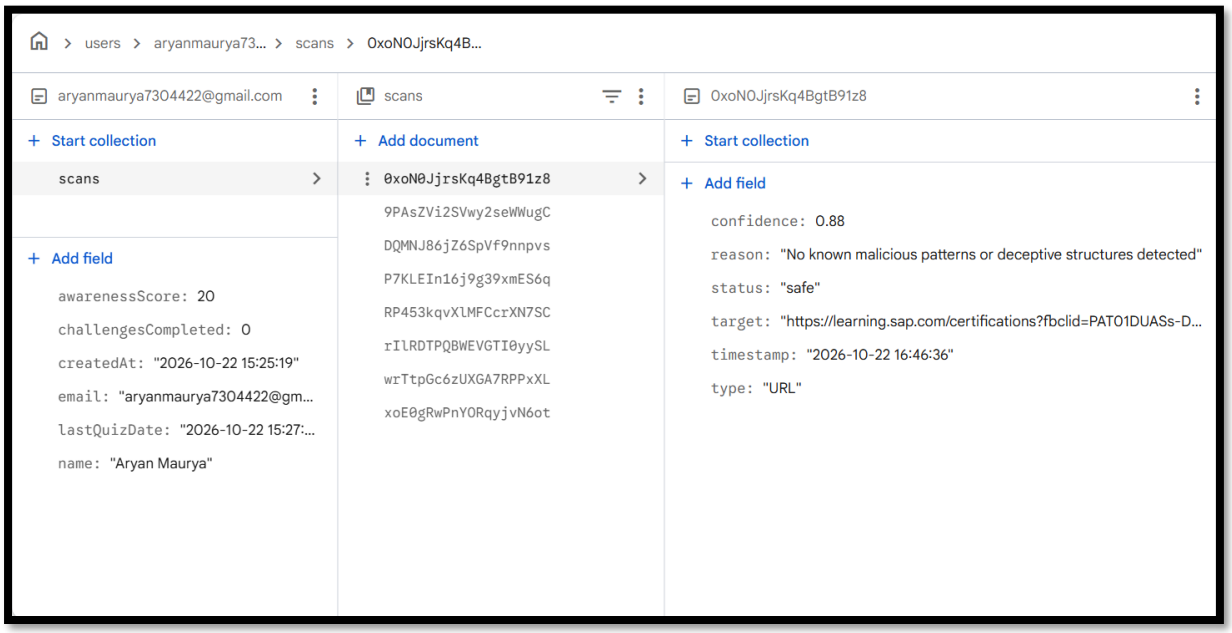



**Description:** The email analysis feature examines the submitted email content to identify possible phishing attempts and digital scams. It checks suspicious wording, urgent requests, embedded URLs, and other warning indicators. The system processes the available email information and displays a result such as SAFE, SUSPICIOUS, or DANGEROUS, along with the reason and security advice when available. This helps users recognise potentially harmful emails before interacting with them.

## 5.2 Database Output




**Description:** AI Cyber Shield uses Firebase Authentication and Cloud Firestore to manage user accounts and store application data. Firestore maintains user profile information, such as name, email, awareness score, and completed challenges. It also stores scan history details, including scan type, target, threat status, confidence, reason, indicators, and timestamp. This database structure helps the application maintain user-related information and retrieve previous scan results.

### 5.3 Discussion

The main purpose of AI Cyber Shield is to help users recognise potential phishing attempts and digital scams through a single Android application. URL scanning, QR code analysis, and SMS and email analysis provide different ways to examine suspicious content.

The combination of machine learning and rule-based checks helps the system evaluate submitted content using the implemented detection logic. The threat result screen communicates the classification and relevant warning indicators, while Firebase supports authentication and cloud storage of user-related information and scan history.

The system's effectiveness depends on the quality of the training data, the detection rules, and the backend response. Some new or modified scams may not be detected correctly. Therefore, the application should be treated as a support tool for cybersecurity awareness rather than a guarantee of complete protection.

# CHAPTER 6:- CONCLUSION AND FUTURE WORK

## 6.1 Conclusion

AI Cyber Shield – Intelligent Multi-Modal Phishing and Digital Scam Protection System is developed to help users identify potential phishing attacks, fraudulent websites, and digital scams. The system analyses URLs, QR codes, SMS messages, and email content to classify potential threats as SAFE, SUSPICIOUS, or DANGEROUS. It uses machine learning and rule-based detection techniques to identify suspicious indicators and provide useful security advice to users.

The application is developed using Java in Android Studio, with a Python Flask backend for content analysis. Firebase Authentication manages user registration and login, while Cloud Firestore stores user profile information and scan history. The system also includes QR code scanning and a Cyber Awareness Challenge to help users understand common online scams and improve their cybersecurity knowledge.

In conclusion, AI Cyber Shield provides a simple and useful platform for improving digital safety and promoting awareness of online threats. It helps users recognise potential risks before interacting with suspicious content and supports safer online decisions. Although the system cannot guarantee the detection of every scam, its features can assist users in identifying possible threats. In the future, improved datasets and more advanced detection techniques can make the system more accurate and effective.

## 6.2 Limitations

1. The accuracy of threat detection depends on the quality and variety of the training dataset.
2. New phishing techniques and unknown scam patterns may not always be detected.
3. Incorrect or incomplete input may affect the analysis result.
4. Internet connectivity is generally required for communication with the Flask backend and cloud services.
5. A SAFE result does not guarantee that the scanned content is completely harmless.

### 6.3 Future Work

1. Improve detection accuracy by training the model with larger and more updated datasets.
2. Add real-time detection of suspicious links received through messages and emails.
3. Support more regional languages to help a wider range of users.
4. Introduce automatic checking of newly reported phishing websites and scam patterns.
5. Improve QR code analysis and add more advanced image-based threat detection.
6. Add regular updates and security awareness content to help users understand new types of digital fraud.

## CHAPTER 7:- CODE IMPLEMENTATION

**Algorithm 1:** Shannon Information Entropy Calculation

- Objective: To detect randomly generated, machine-created phishing domains, also known as Domain Generation Algorithm (DGA) domains.
- Mathematical Formula: \[ H(X)=-\sum P(x_i)\cdot\log_2(P(x_i)) \]

**Python code:**

```python
import math

def calculate_entropy(text: str) -> float:
    if not text:
        return 0.0
    entropy = 0.0
    length = len(text)
    for char in set(text):
        p_x = text.count(char) / length
        if p_x > 0:
            entropy -= p_x * math.log2(p_x)
    return round(entropy, 3)
```

**Algorithm 2:** URL Structural Parsing & Brand Spoofing Detection

- Objective: To detect phishing URLs that impersonate trusted brands such as PayPal, Chase, and Apple by placing their names in subdomains or URL paths.
- Core Rule: If a trusted brand name appears in the subdomain or URL path but does not match the root domain, the URL is classified as HIGH-RISK SPOOFING.

**Python Code:**

```python
import re
from urllib.parse import urlparse
import tldextract

def detect_brand_spoofing_and_structure(url: str, targeted_brands: list) -> dict:
    parsed = urlparse(url if url.startswith(('http://', 'https://')) else 'http://' + url)
    ext = tldextract.extract(url)

    root_domain = ext.domain.lower()
    subdomain_and_path = (ext.subdomain + " " + parsed.path).lower()
    netloc = parsed.netloc.lower()

    # 1. IP Address Host Check
    has_ip_host = bool(re.match(r'^(\d{1,3}\.){3}\d{1,3}(:\d+)?$', netloc))

    # 2. Brand Spoofing Check
    is_brand_spoofed = False
    spoofed_brands = []
    for brand in targeted_brands:
        if brand in subdomain_and_path and brand != root_domain:
            is_brand_spoofed = True
            spoofed_brands.append(brand)
    return {
        "has_ip_host": has_ip_host,
        "is_brand_spoofed": is_brand_spoofed,
        "spoofed_brands": spoofed_brands,
        "is_https": parsed.scheme == 'https'
    }
```

**Algorithm 3:** Hybrid Multi-Tier AI + Heuristic Threat Decision Matrix

- Objective: To serve as the main decision-making engine of the system by combining Machine Learning (Random Forest) probability scores with deterministic security rules to generate a final threat verdict.

**Python Code:**

```python
def hybrid_threat_decision_engine(features: dict, ml_model, raw_url: str) -> dict:
    # 1. Machine Learning Inference
    ml_label = 'safe'
    ml_confidence = 0.85
    if ml_model:
        probs = ml_model.predict_proba([raw_url])[0]
        max_idx = probs.argmax()
        ml_label = ml_model.classes_[max_idx]
        ml_confidence = float(probs[max_idx])

    # 2. Critical Heuristic Trigger
    has_critical_red_flags = (
        features['has_ip_host'] or
        features['is_brand_spoofed'] or
        (features['has_suspicious_tld'] and features['keyword_count'] >= 1)
    )

    # 3. Hybrid Arbitration Matrix
    if features.get('is_trusted_authority', False) and not features['is_brand_spoofed']:
        verdict = 'safe'
        confidence = max(0.95, ml_confidence)
    elif has_critical_red_flags or ml_label == 'dangerous':
        verdict = 'dangerous'
        confidence = max(0.92, ml_confidence)
    elif features.get('is_shortener', False) or ml_label == 'suspicious':
        verdict = 'suspicious'
        confidence = max(0.78, ml_confidence)
    else:
        verdict = 'safe'
        confidence = max(0.88, ml_confidence)

    return {"status": verdict, "confidence": round(confidence, 2)}
```